Fiche Exercice : Système de Gestion d'Utilisateurs Sécurisé

Objectif : Créer un portail complet permettant l'inscription, la connexion sécurisée et la gestion (CRUD) d'utilisateurs avec PHP et MySQL.

Action de l'utilisateur	Fichier de départ	Méthode	Fichier de traitement	Résultat en Base de données	Redirection finale
S'inscrire	inserer.html	POST	connect.php	Crée une nouvelle ligne (MDP haché)	index.html
Se connecter	login.php	POST	login.php	Aucune (Lecture seule)	liste_utilisateurs.php
Se déconnecter	Lien Nav	-	logout.php	Aucune (Détruit la Session)	index.html
Modifier un profil	modifier.php	POST	modifier.php	Met à jour une ligne existante	liste_utilisateurs.php
Supprimer un client	Lien Liste	GET	supprimer.php	Efface définitivement une ligne	liste_utilisateurs.php
Sécuriser la base	Navigateur	-	hacher_mdp.php	Transforme les MDP en clair en Hash	login.php

Les Phases de Réalisation (Étape par Étape)

Phase A : L'Inscription (`inserer.html` → `connect.php`)

1. **Côté HTML :** Créer un formulaire avec `method="POST"`. Utiliser `type="password"` pour le mot de passe.

Fiche Exercice : Système de Gestion d'Utilisateurs Sécurisé

Objectif : Créer un portail complet permettant l'inscription, la connexion sécurisée et la gestion (CRUD) d'utilisateurs avec PHP et MySQL.

2. **Côté PHP :** * Récupérer les données avec `$_POST`.
    - **Sécurité :** Utiliser `$hash = password_hash($pwd, PASSWORD_DEFAULT)`.
    - **BDD :** Faire un `INSERT INTO` avec une requête préparée (`prepare` / `execute`).

Phase B : La Connexion (`login.php`)

1. **Recherche :** Récupérer l'utilisateur en BDD par son nom unique.
2. **Vérification :** Utiliser `password_verify($saisie, $hash_bdd)`.
3. **Session :** Si OK, remplir `$_SESSION['user_id']` et `$_SESSION['role']`.

Phase C : La Protection (`liste_utilisateurs.php`)

1. **Contrôle d'accès :** Vérifier en haut de page si `$_SESSION['user_id']` existe. Sinon ➔ Redirection.
2. **Affichage :** Faire un `SELECT *` et boucler (`while`) pour générer les lignes du tableau.
3. **Liens :** Ajouter des liens dynamiques : `modifier.php?id=<?php echo $row['id']; ?>`.

Phase D : La Modification (`modifier.php`)

1. **Réception (GET) :** Récupérer l'ID dans l'URL pour afficher les anciennes données dans les `value` du formulaire.
2. **Envoi (POST) :** Utiliser un `<input type="hidden">` pour conserver l'ID lors de l'envoi.
3. **Mise à jour :** Exécuter une requête `UPDATE clients SET ... WHERE id = ?`.

Phase E : La Maintenance (`hacher_mdp.php`)

1. **Boucle :** Parcourir toute la table `clients`.
2. **Test :** Vérifier si le mot de passe commence par `$2y$`.

Fiche Exercice : Système de Gestion d'Utilisateurs Sécurisé

Objectif : Créer un portail complet permettant l'inscription, la connexion sécurisée et la gestion (CRUD) d'utilisateurs avec PHP et MySQL.

3. **Action :** Si non, le hacher et mettre à jour la ligne.

Fichier	Type	Rôle Principal
index.html	Entrée	Page d'accueil avec liens vers Connexion ou Inscription.
inserer.html	Formulaire	Formulaire de création de compte (User, Pwd, CP, Auth).
connect.php	Traitement	Reçoit les données de inserer.html, **hache le MDP** et insère en BDD.
login.php	Mixte	Affiche le formulaire et vérifie les identifiants via password_verify.
logout.php	Action	Détruit la session en cours et redirige vers l'accueil.
liste_utilisateurs.php	Affichage	Affiche le tableau des membres (sécurisé par session).
modifier.php	Mixte	Récupère les données d'UN utilisateur et permet sa mise à jour.
hacher_mdp.php	Maintenance	Script "One-Shot" pour sécuriser les anciens mots de passe en clair.

Fiche Exercice : Système de Gestion d'Utilisateurs Sécurisé

Objectif : Créer un portail complet permettant l'inscription, la connexion sécurisée et la gestion (CRUD) d'utilisateurs avec PHP et MySQL.

Le concept du "CRUD" (Aide-mémoire)

Le **CRUD**, l'acronyme de base de tout développeur web :

- **C**reate (Créer) ➔ `connect.php`
- **R**ead (Lire) ➔ `liste_utilisateurs.php`
- **U**pdate (Mettre à jour) ➔ `modifier.php`
- **D**elete (Supprimer) ➔ `supprimer.php`

Points de Vigilance :

- **Sécurité XSS :** Toujours utiliser `htmlspecialchars()` lors de l'affichage de données venant de la BDD.
- **Requêtes Préparées :** Ne jamais injecter de variables directement dans la chaîne SQL (utiliser les `?`).
- **Session :** Ne jamais oublier `session_start()` au tout début de chaque fichier PHP utilisant les sessions.
- **Expérience Utilisateur :** Toujours rediriger l'utilisateur après une action POST (`header("Location: ...")`) pour éviter les renvois de formulaires involontaires au rafraîchissement de la page.

Fiche Exercice : Système de Gestion d'Utilisateurs Sécurisé

Objectif : Créer un portail complet permettant l'inscription, la connexion sécurisée et la gestion (CRUD) d'utilisateurs avec PHP et MySQL.

Le Lexique du Développeur PHP

Ce lexique regroupe les fonctions essentielles utilisées dans votre projet de gestion d'utilisateurs.

🔐 Sécurité & Mots de passe

- **`password_hash($mdp, PASSWORD_DEFAULT)`** : Transforme un mot de passe clair en une empreinte illisible (le "hash"). C'est la seule façon sécurisée de stocker des mots de passe.
- **`password_verify($saisie, $hash_bdd)`** : Compare le mot de passe tapé par l'utilisateur avec l'empreinte stockée en base. Elle renvoie `true` (vrai) ou `false` (faux).
- **`htmlspecialchars($texte)`** : Neutralise les balises HTML. Elle empêche les pirates d'injecter du code malveillant dans vos pages (protection contre les failles XSS).

👥 Gestion des Sessions

- **`session_start()`** : À placer tout en haut de chaque fichier. Elle ouvre ou récupère une session sur le serveur, permettant de "se souvenir" de l'utilisateur (son nom, son rôle) sur toutes les pages.
- **`$_SESSION`** : Un tableau superglobal qui contient les informations de l'utilisateur connecté.
- **`session_destroy()`** : Supprime définitivement la session côté serveur (utilisée dans `logout.php`).

🗄️ Base de données (PDO)

- **`new PDO(...)`** : Crée la connexion entre PHP et MySQL.
- **`prepare($sql)`** : Prépare une requête SQL avec des paramètres (les `?`). C'est un bouclier contre les injections SQL (piratage de la base).
- **`execute([$valeurs])`** : Envoie réellement les données à MySQL pour exécuter la requête préparée.
- **`fetch()`** / **`fetchAll()`** : "Va chercher" les résultats de la requête pour les transformer en tableau PHP utilisable.

Fiche Exercice : Système de Gestion d'Utilisateurs Sécurisé

Objectif : Créer un portail complet permettant l'inscription, la connexion sécurisée et la gestion (CRUD) d'utilisateurs avec PHP et MySQL.

🚀 Navigation & Flux

- **`header("Location: page.php")`** : Envoie un ordre au navigateur pour changer de page immédiatement (Redirection).
- **`exit()`** : Arrête immédiatement l'exécution du code PHP. Indispensable après une redirection pour éviter que la suite du script ne s'exécute pour rien.
- **`isset($variable)`** : Vérifie si une variable existe (par exemple, si un formulaire a bien été envoyé ou si un ID est présent dans l'URL).